

NBP Data Analysis Tool PROJECT DOCUMENTATION

1. System architecture

The system is designed as a monolithic, single-process application running locally on user's machine as a console application developed in Python. The main goal of the application is to fetch, model and perform statistical analysis on currency exchange data provided by the National Bank of Poland (NBP) API.

The application is divided into layers ensuring separation of concerns:

- *Interaction layer* – operates within a continuous runtime loop where it captures user requirements and execution parameters. It actively communicates across all architectural layers to orchestrate data acquisition, statistical computation, and output formatting.
- *Presentation layer* – it encompasses the command-line interface logic and ensures that the output is formatted clearly. Its primary responsibilities include input sanitization (capturing and validating user commands) and output formatting (transforming unclear statistical data into human-readable text).
- *Infrastructure layer* – this layer is mainly responsible for fetching the requested data from external services (NBP API) via HTTP. The implementation of this layer abstracts and manages HTTP requests and responses, dynamic URL construction. It also ensures that the application stays functional even if the connection fails.
- *Business logic layer* – the core of the project, responsible for calculating requested by the user measures using raw, statistical data fetched from the NBP API. Its main responsibilities include session analysis (computing rising, losing and flat sessions), computing statistical measures (median, mode, standard deviation and coefficient of variation) and calculating distribution of changes for a given pair of currencies.

2. CI/CD

The development lifecycle of the system is governed by an automated continuous integration and continuous delivery (CI/CD) pipeline built using GitHub Actions.

Continuous integration workflow is executed after a push to *develop*, *release* or *main* branch or after a pull request to these branches. The workflow executes the following steps:

- runs a Python runtime environment,
- installs dependencies specified in *requirements.txt* file,
- checks for code linting and formatting issues,
- runs unit tests.

If everything succeeds, a push or merge becomes possible.

Continuous delivery workflow is executed after the continuous integration workflow finishes and only if a new tag has been pushed. It executes the following steps:

- runs a Python runtime environment,
- installs dependencies specified in *requirements.txt* file,
- builds an executable file (.exe, exclusive to Windows)
- pushes a new release on GitHub and attaches the executable as an artifact.